

Argon2

Designers: Alex Biryukov, Daniel Dinu, and Dmitry Khovratovich
University of Luxembourg, Luxembourg

`alex.biryukov@uni.lu, dmitry.khovratovich@uni.lu, khovratovich@gmail.com`

`https://www.cryptolux.org/index.php/Argon2
https://github.com/khovratovich/Argon2`

Version 1.2 of Argon2

July 8, 2015

Contents

1	Introduction	2
2	Argon2	4
2.1	Specification	4
2.1.1	Inputs	4
2.1.2	Operation	5
2.1.3	Indexing	5
2.1.4	Compression function G	6
2.2	Features	7
2.2.1	Available features	7
2.2.2	Server relief	8
2.2.3	Client-independent update	9
2.2.4	Possible future extensions	9
2.3	Design rationale	9
2.3.1	Cost estimation	9
2.3.2	Mode of operation	10
2.3.3	Implementing parallelism	10
2.3.4	Compression function design	11
2.3.5	User-controlled parameters	13
2.4	Security analysis	13
2.4.1	Security of single-pass schemes	13
2.4.2	Ranking tradeoff attack	14
2.4.3	Multi-pass schemes	15
2.4.4	Security of Argon2 to generic attacks	15
2.4.5	Security of Argon2 to tradeoff attacks	16
2.5	Performance	17
2.5.1	x86 architecture	17
2.6	Applications	17
2.7	Other details	18
2.7.1	Permutation $\mathcal{P}$	18
2.7.2	Proof of Lemma 1	18
3	Update	20
3.1	Indexing	20
3.1.1	Description	20
3.1.2	Rationale	20
3.2	New compression function	21
3.2.1	Design	21
3.2.2	Rationale	22
3.3	Performance	22
4	Change log	24
4.1	v1.2 of Argon2 – 21th June, 2015	24
4.2	v1.1 of Argon2 – 4th February, 2015	24

Chapter 1

Introduction

Passwords remain the primary form of authentication on various web-services. Passwords are usually stored in a hashed form in a server's database. These databases are quite often captured by the adversaries, who then apply dictionary attacks since passwords tend to have low entropy. Protocol designers use a number of tricks to mitigate these issues. Starting from the late 70's, a password is hashed together with a random *salt* value to prevent detection of identical passwords across different users and services. The hash function computations, which became faster and faster due to Moore's law have been called multiple times to increase the cost of password trial for the attacker.

In the meanwhile, the password crackers migrated to new architectures, such as FPGAs, multiple-core GPUs and dedicated ASIC modules, where the amortized cost of a multiple-iterated hash function is much lower. It was quickly noted that these new environments are great when the computation is almost memoryless, but they experience difficulties when operating on a large amount of memory. The defenders responded by designing *memory-hard* functions, which require a large amount of memory to be computed. The password hashing scheme `scrypt` [17] is an instance of such function.

Password hashing schemes also have other applications. They can be used for key derivation from low-entropy sources. Memory-hard schemes are also welcome in cryptocurrency designs [4] if a creator wants to demotivate the use of GPUs and ASICs for mining and promote the use of standard desktops.

Problems of existing schemes. A design of a memory-hard function proved to be a tough problem. Since early 80's it has been known that many cryptographic problems that seemingly require large memory actually allow for a time-memory tradeoff [13], where the adversary can trade memory for time and do his job on fast hardware with low memory. In application to password-hashing schemes, this means that the password crackers can still be implemented on a dedicated hardware even though at some additional cost. The `scrypt` function, for example, allows for a simple tradeoff where the time-area product remains almost constant.

Another problem with the existing schemes is their complexity. The same `scrypt` calls a stack of subprocedures, whose design rationale has not been fully motivated (e.g, `scrypt` calls `SMix`, which calls `ROMix`, which calls `BlockMix`, which calls `Salsa20/8` etc.). It is hard to analyze and, moreover, hard to achieve confidence. Finally, it is not flexible in separating time and memory costs. At the same time, the story of cryptographic competitions [2, 20, 3] has demonstrated that the most secure designs come with simplicity, where every element is well motivated and a cryptanalyst has as few entry points as possible.

The ongoing Password Hashing Competition also highlighted the following problems:

- Should the memory addressing (indexing functions) be input-independent or input-dependent, or hybrid? The first type of schemes, where the memory read location are known in advance, is immediately vulnerable to time-space tradeoff attacks, since an adversary can precompute the missing block by the time it is needed [7]. In turn, the input-dependent schemes are vulnerable to side-channel attacks [19], as the timing information allows for much faster password search.
- Is it better to fill more memory but suffer from time-space tradeoffs, or make more passes over the memory to be more robust? This question was quite difficult to answer due to absence of generic tradeoff tools, which would analyze the security against tradeoff attacks, and the absence of unified metric to measure adversary's costs.
- How should the input-independent addresses be computed? Several seemingly secure options have been attacked [7].

- How large a single memory block should be? Reading smaller random-placed blocks is slower (in cycles per byte) due to the spacial locality principle of the CPU cache. In turn, larger blocks are difficult to process due to the limited number of long registers.
- If the block is large, how to choose the internal compression function? Should it be cryptographically secure or more lightweight, providing only basic mixing of the inputs? Many candidates simply proposed an iterative construction and argued against cryptographically strong transformations.
- Which operations should be used by the compression function in the goal of maximizing adversary's costs with fixed desktop performance? Should we use recent AES instructions, ARX primitives, integer and floating-point operations?
- How to exploit multiple cores of modern CPUs, when they are available? Parallelizing calls to the hashing function without any interaction is subject to simple tradeoff attacks.

Our solutions. We offer a hashing scheme called **Argon2**. **Argon2** summarizes the state of the art in the design of memory-hard functions. It is a streamlined and simple design. It aims at the highest memory filling rate and effective use of multiple computing units, while still providing defense against tradeoff attacks. **Argon2** is optimized for the x86 architecture and exploits the cache and memory organization of the recent Intel and AMD processors. **Argon2** has two variants: **Argon2d** and **Argon2i**. **Argon2d** is faster and uses data-depending memory access, which makes it suitable for cryptocurrencies and applications with no threats from side-channel timing attacks. **Argon2i** uses data-independent memory access, which is preferred for password hashing and password-based key derivation. **Argon2i** is slower as it makes more passes over the memory to protect from tradeoff attacks.

We recommend **Argon2** for the applications that aim for high performance. Both versions of **Argon2** allow to fill 1 GB of RAM in a fraction of second, and smaller amounts even faster. It scales easily to the arbitrary number of parallel computing units. Its design is also optimized for clarity to ease analysis and implementation.

Chapter 2

Argon2

In this chapter we introduce the next generation of memory-hard hash function **Argon**, which we call **Argon2**. **Argon2** is built upon the most recent research in the design and analysis of memory-hard functions. It is suitable for password hashing, password-based key derivation, cryptocurrencies, proofs of work/space, etc.

2.1 Specification

There are two flavors of **Argon2** – **Argon2d** and **Argon2i**. The former one uses data-dependent memory access to thwart tradeoff attacks. However, this makes it vulnerable for side-channel attacks, so **Argon2d** is recommended primarily for cryptocurrencies and backend servers. **Argon2i** uses data-independent memory access, which is recommended for password hashing and password-based key derivation.

2.1.1 Inputs

Argon2 has two types of inputs: primary inputs and secondary inputs, or parameters. Primary inputs are message P and nonce S , which are password and salt, respectively, for the password hashing. Primary inputs must always be given by the user such that

- Message P may have any length from 0 to $2^{32} - 1$ bytes;
- Nonce S may have any length from 8 to $2^{32} - 1$ bytes (16 bytes is recommended for password hashing).

Secondary inputs have the following restrictions:

- Degree of parallelism p may take any integer value from 1 to 64.
- Tag length τ may be any integer number of bytes from 4 to $2^{32} - 1$.
- Memory size m can be any integer number of kilobytes from $8p$ to $2^{32} - 1$, but it is rounded down to the nearest multiple of $4p$.
- Number of iterations t can be any integer number from 1 to $2^{32} - 1$;
- Version number v is one byte $0x10$;
- Secret value K may have any length from 0 to 32 bytes.
- Associated data X may have any length from 0 to $2^{32} - 1$ bytes.

Argon2 uses compression function G and hash function H . Here H is the Blake2b hash function, and G is based on its internal permutation. The mode of operation of **Argon2** is quite simple when no parallelism is used: function G is iterated m times. At step i a block with index $\phi(i) < i$ is taken from the memory (Figure 2.1), where $\phi(i)$ is either determined by the previous block in **Argon2d**, or is a fixed value in **Argon2i**.

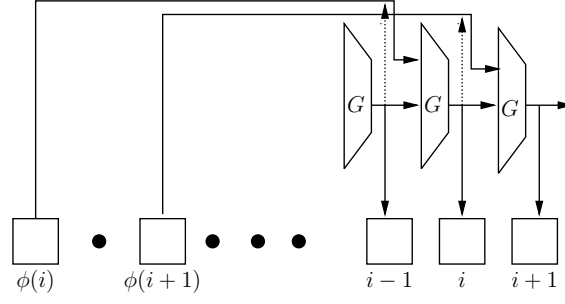


Figure 2.1: Argon2 mode of operation with no parallelism.

2.1.2 Operation

Argon2 follows the extract-then-expand concept. First, it extracts entropy from message and nonce by hashing it. All the other parameters are also added to the input. The variable length inputs P and S are prepended with their lengths:

$$H_0 = \mathcal{H}(d, \tau, m, t, v, \langle P \rangle, P, \langle S \rangle, S, \langle K \rangle, K, \langle X \rangle, X).$$

Here H_0 is 32-byte value.

Argon2 then fills the memory with m 1024-byte blocks. For tunable parallelism with p threads, the memory is organized in a matrix $B[i][j]$ of blocks with p rows (*lanes*) and $q = \lfloor m/d \rfloor$ columns. Blocks are computed as follows:

$$\begin{aligned} B[i][0] &= G(H_0, \underbrace{i}_{4 \text{ bytes}} \parallel \underbrace{0}_{4 \text{ bytes}}), \quad 0 \leq i < p; \\ B[i][1] &= G(H_0, \underbrace{i}_{4 \text{ bytes}} \parallel \underbrace{1}_{4 \text{ bytes}}), \quad 0 \leq i < p; \\ B[i][j] &= G(B[i][j-1], B[i'][j']), \quad 0 \leq i < p, \quad 2 \leq j < q. \end{aligned}$$

where block index $[i'][j']$ is determined differently for Argon2d and Argon2i, and G is the compression function. Both will be fully defined in the further text. The inputs to G that are not full blocks, are prepended by necessary number of zeros.

If $t > 1$, we repeat the procedure; however the first two columns are now computed in the same way:

$$\begin{aligned} B[i][0] &= G(B[i][q-1], B[i'][j']); \\ B[i][j] &= G(B[i][j-1], B[i'][j']). \end{aligned}$$

When we have done t iterations over the memory, we compute the final block B_m as the XOR of the last column:

$$B_m = B[0][q-1] \oplus B[1][q-1] \oplus \dots \oplus B[p-1][q-1].$$

The output tag is produced as follows. The hash function $\mathcal{H}$ is applied iteratively to B_m , each time outputting the first 32 bytes of the 64-byte hash value, until the total number of output bytes reaches τ .

2.1.3 Indexing

Now we explain how the index $[i'][j']$ of the reference block is computed. First, we determine the set of indices $\mathcal{R}$ that can be referenced for given $[i][j]$. For that we further partition the memory matrix into $l = 4$ vertical *slices*. Intersection of a slice and a lane is *segment* of length q/l . Thus segments form their own matrix Q of size $p \times l$. Segments of the same slice are computed in parallel, and may not reference blocks from each other. All other blocks can be referenced. Suppose we are computing a block in a segment $Q[r][s]$. Then $\mathcal{R}$ includes blocks according to the following rules:

1. All blocks of segments $Q[r'][*]$, where $r' < r$ and $*$ takes all possible values from 0 to $p-1$.
2. All blocks of segments $Q[r'][*]$, where $r' > r$ and $*$ takes all possible values from 0 to $p-1$ — if it is the second or later pass over the memory.

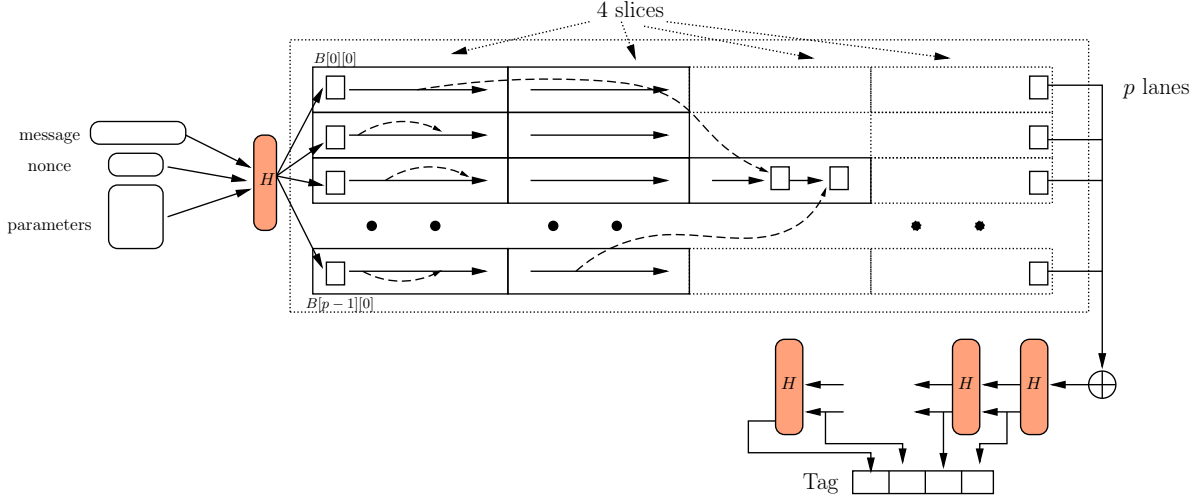


Figure 2.2: Single-pass Argon2 with p lanes and 4 slices.

3. All blocks of segment $Q[r][s]$ (current segment) except for the previous block;
4. For the first block of the segment $\mathcal{R}$ does not include the previous block of any lane.

Let R be the size of $\mathcal{R}$. The blocks in $\mathcal{R}$ are numbered from 0 to $R - 1$ according to the following rules:

- Blocks outside of the current segment are enumerated first;
- Segments are enumerated from top to down, and then from left to right: $Q[0][0]$, then $Q[1][0]$, then up to $Q[p - 1][0]$, then $Q[0][1]$, then $Q[1][1]$, etc.
- Blocks within a segment are enumerated from the oldest to the newest.

Then Argon2d selects a block from $\mathcal{R}$ basing on the previous block (thus depending on the password), whereas Argon2i selects it independently of the input:

- In Argon2d we take the first 32 bits of block $B[i][j - 1]$ and denote this block by J . Then the value $J \pmod{R}$ determines the block number from $\mathcal{R}$.
- In Argon2i we run G^2 — the 2-round compression function G — in the counter mode, where the first input is all-zero block, and the second input is constructed as

$$\left(\underbrace{r}_{4 \text{ bytes}} \parallel \underbrace{l}_{4 \text{ bytes}} \parallel \underbrace{s}_{4 \text{ bytes}} \parallel \underbrace{i}_{4 \text{ bytes}} \parallel \underbrace{0}_{1008 \text{ bytes}} \right),$$

where r is the pass number, l is the lane, s is the slice, and i is the counter starting in each segment from 0. Then we increase the counter so that each application of G^2 gives 256 values of J , which are used to reference available blocks exactly as in Argon2d.

2.1.4 Compression function G

Compression function G is built upon the Blake2b round function $\mathcal{P}$ (fully defined in Section 2.7.1). $\mathcal{P}$ operates on the 128-byte input, which can be viewed as 8 16-byte registers (see details below):

$$\mathcal{P}(A_0, A_1, \dots, A_7) = (B_0, B_1, \dots, B_7).$$

Compression function $G(X, Y)$ operates on two 1024-byte blocks X and Y . It first computes $R = X \oplus Y$. Then R is viewed as a 8×8 -matrix of 16-byte registers $R_0, R_1, \dots, R_{63}$. Then $\mathcal{P}$ is first applied

rowwise, and then columnwise to get Z :

$$\begin{aligned}
(Q_0, Q_1, \dots, Q_7) &\leftarrow \mathcal{P}(R_0, R_1, \dots, R_7); \\
(Q_8, Q_9, \dots, Q_{15}) &\leftarrow \mathcal{P}(R_8, R_9, \dots, R_{15}); \\
&\dots \\
(Q_{56}, Q_{57}, \dots, Q_{63}) &\leftarrow \mathcal{P}(R_{56}, R_{57}, \dots, R_{63}); \\
(Z_0, Z_8, Z_{16}, \dots, Z_{56}) &\leftarrow \mathcal{P}(Q_0, Q_8, Q_{16}, \dots, Q_{56}); \\
(Z_1, Z_9, Z_{17}, \dots, Z_{57}) &\leftarrow \mathcal{P}(Q_1, Q_9, Q_{17}, \dots, Q_{57}); \\
&\dots \\
(Z_7, Z_{15}, Z_{23}, \dots, Z_{63}) &\leftarrow \mathcal{P}(Q_7, Q_{15}, Q_{23}, \dots, Q_{63}).
\end{aligned}$$

Finally, G outputs $Z \oplus R$:

$$G : (X, Y) \rightarrow R = X \oplus Y \xrightarrow{\mathcal{P}} Q \xrightarrow{\mathcal{P}} Z \rightarrow Z \oplus R.$$

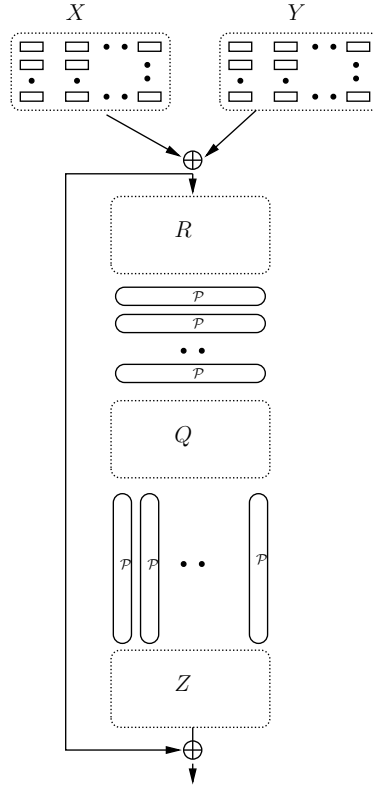


Figure 2.3: Argon2 compression function G .

2.2 Features

Argon2 is a multi-purpose family of hashing schemes, which is suitable for password hashing, key derivation, cryptocurrencies and other applications that require provably high memory use. Argon2 is optimized for the x86 architecture, but it does not slow much on older processors. The key feature of Argon2 is its performance and the ability to use multiple computational cores in a way that prohibit time-memory tradeoffs. Several features are not included into this version, but can be easily added later.

2.2.1 Available features

Now we provide an extensive list of features of Argon.

Design rationality	Argon2 follows a simple design with three main ideas: generic mode of operation, tradeoff-resilient parallelism with synchronization points, and tradeoff-resistant compression function. All design choices are based on the intensive research and explained below.
Performance	Argon2 fills memory very fast. Data-independent version Argon2i securely fills the memory spending about a CPU cycle per byte, and Argon2d is twice as fast. This makes it suitable for applications that need memory-hardness but can not allow much CPU time, like cryptocurrency peer software.
Tradeoff resilience	Despite high performance, Argon2 provides reasonable level of tradeoff resilience. Our tradeoff attacks previously applied to Catena and Lyra2 show the following. With default number of passes over memory (1 for Argon2d, 3 for Argon2i, an ASIC-equipped adversary can not decrease the time-area product if the memory is reduced by the factor of 4 or more. Much higher penalties apply if more passes over the memory are made.
Scalability	Argon2 is scalable both in time and memory dimensions. Both parameters can be changed independently provided that a certain amount of time is always needed to fill the memory.
Parallelism	Argon2 may use up to 64 threads in parallel, although in our experiments 8 threads already exhaust the available bandwidth and computing power of the machine.
Server relief	Argon2 allows the server to carry out the majority of computational burden on the client in case of denial-of-service attacks or other situations. The server ultimately receives a short intermediate value, which undergoes a preimage-resistant function to produce the final tag.
Client-independent updates	Argon2 offers a mechanism to update the stored tag with an increased time and memory cost parameters. The server stores the new tag and both parameter sets. When a client logs into the server next time, Argon is called as many times as need using the intermediate result as an input to the next call.
Possible extensions	Argon2 is open to future extensions and tweaks that would allow to extend its capabilities. Some possible extensions are listed in Section 2.2.4.
GPU/FPGA/ASIC-unfriendly	Argon2 is heavily optimized for the x86 architecture, so that implementing it on dedicated cracking hardware should be neither cheaper nor faster. Even specialized ASICs would require significant area and would not allow reduction in the time-area product.
Secret input support	Argon2 natively supports secret input, which can be <i>key</i> or any other secret string.
Associated data support	Argon2 supports additional input, which is syntactically separated from the message and nonce, such as environment parameters, user data, etc..

2.2.2 Server relief

The server relief feature allows the server to delegate the most expensive part of the hashing to the client. This can be done as follows (suppose that K and X are not used):

1. The server communicates S, m, R, d, τ to the client.
2. The client performs all computations up to the value of (B_0, B_1) ;
3. The client communicates (B_0, B_1) to the server;
4. The server computes Tag and stores it together with S, m, d, R .

The tag computation function is preimage-resistant, as it is an iteration of a cryptographic hash function. Therefore, leaking the tag value would not allow the adversary to recover the actual password nor the value of B by other means than exhaustive search.

2.2.3 Client-independent update

It is possible to increase the time and memory costs on the server side without requiring the client to re-enter the original password. We just compute

$$\text{Tag}_{new} = \Pi(\text{Tag}_{old}, S, m_{new}, R_{new}, \tau_{new}),$$

replace Tag_{old} with Tag_{new} in the hash database and add the new values of m and R to the entry.

2.2.4 Possible future extensions

Argon can be rather easily tuned to support the following extensions, which are not used now for design clarity or performance issues:

Support for other hash functions	Any cryptographically strong hash function may replace Blake2b for the initial and the final phases. However, if it does not support variable output length natively, some care should be taken when implementing it.
Support for other compression functions and block sizes	Compression function can be changed to any other transformation that fulfills our design criteria. It may use shorter or longer blocks.
Support for Read-Only-Memory	ROM can be easily integrated into Argon2 by simply including it into the area where the blocks are referenced from.

2.3 Design rationale

Argon2 was designed with the following primary goal: to maximize the cost of exhaustive search on non-x86 architectures, so that the switch even to dedicated ASICs would not give significant advantage over doing the exhaustive search on defender’s machine.

2.3.1 Cost estimation

We have investigated several approaches to estimating the real cost of the brute-force. As the first approximation, we calculate the hardware design&production costs separately from the actual running costs. The choice of the platform is determined by adversary’s budget: if it is below 100000 EUR or so, then GPU and/or FPGA are the reasonable choices, whereas attackers with a million budget (governments or their subsidiaries) can afford development and production of the dedicated hardware (ASIC).

If ASICs are not considered, the hashing schemes that use more than a few hundred MB of RAM are almost certainly inefficient on a GPU or a FPGA. GPUs efficiency gain (compared to regular desktops) is rather moderate even on low-memory computations, which can be parallelized. If we consider a scheme with a low degree of parallelism and intensive memory use, then the high memory latency of the GPU architecture would make the brute-force much less efficient.

Argon2 protects against all types of adversaries, including those who can afford design and production of dedicated hardware for password cracking or cryptocurrency mining. However, modelling the attack cost is not easy. Our first motivation was to calculate the exact running costs [7] by outlining a hypothetical brute-force chip, figuring out the energy consumption of all components and calculate the total energy-per-hash cost. However, we found out that the choice of the memory type and assumptions on its properties affect the final numbers significantly. For example, taking a low-leakage memory [21] results in the underestimation of the real power consumption, because we do not know exactly how it scales exactly with increasing the size (the paper [21] proposes a 32-KB chip). On the other hand, the regular DDR3 as a reference also creates ambiguity, since it is difficult to estimate its idle power consumption [1].

To overcome these problems, we turn to a more robust metric of the time-area product [22, 6]. In the further text, we’ll be interested in the area requirements by memory, by the Blake2 core [5], and the maximum possible bandwidth existing in commercial products.

- The 50-nm DRAM implementation [11] takes 550 mm² per GByte;
- The Blake2b implementation in the 65-nm process should take about 0.1 mm² (using Blake-512 implementation in [12]);
- The maximum memory bandwidth achieved by modern GPUs is around 400 GB/sec.

2.3.2 Mode of operation

We decided to make a simple mode of operation. It is based on the single-pass, single-thread filling memory array $B[]$ with the help of the compression function G :

$$\begin{aligned} B[0] &= H(P, S); \\ \text{for } j &\text{ from 1 to } t \\ B[j] &= G(B[\phi_1(j)], B[\phi_2(j)], \dots, B[\phi_k(j)]), \end{aligned} \tag{2.1}$$

where $\phi_i()$ are some indexing functions.

This scheme was extended to implement:

- Tunable parallelism;
- Several passes over memory.

Number of referenced blocks. The number k is a tradeoff between performance and bandwidth. First, we set $\phi_k(j) = j - 1$ to maximize the total chain length. Then we decide on the exact value of k . One might expect that this value should be determined by the maximum L1 cache bandwidth. Two read ports at the Haswell CPU should have allowed to spend twice fewer cycles to read than to write. In practice, however, we do not get such speed-up with our compression function:

k	Cycles/block	Bandwidth (GB/sec)
2	1194	4.3
3	1503	5.1

Thus extra block decreases the speed but increases the bandwidth. Larger values of k would further reduce the throughput and thus the total amount of memory filled in the fixed time. However, an adversary will respond by creating a custom CPU that will be marginally larger but having k load ports, thereby keeping the time the same. Thus we conclude that the area-time product of attacker's hardware is maximized by $k = 2$ or $k = 3$, depending on the compression function. We choose $k = 2$ as we aim for faster memory fill and more passes over the memory.

Number of output blocks. We argue that only one output block should be produced per step if we use $k = 2$. An alternative would be to have some internal state of size $t' \geq t$ and to iterate F several times (say, l') with outputting a t -bit block every round. However, it can be easily seen that if $l't < t'$, then an adversary could simply keep the state in the memory instead of the blocks and thus get a free tradeoff. If $l't = t'$, then, in turn, an adversary would rather store the two input blocks.

Indexing functions. For the data-dependent addressing we set $\phi(l) = g(B[l])$, where g simply truncates the block and takes the result modulo $l - 1$. We considered taking the address not from the block $B[l - 1]$ but from the block $B[l - 2]$, which should have allowed to prefetch the block earlier. However, not only the gain in our implementations is limited, but also this benefit can be exploited by the adversary. Indeed, the efficient depth $D(q)$ is now reduced to $D(q) - 1$, since the adversary has one extra timeslot. Table 2.1 implies that then the adversary would be able to reduce the memory by the factor of 5 without increasing the time-area product (which is a 25% increase in the reduction factor compared to the standard approach).

For the data-independent addressing we use a simple PRNG, in particular the compression function G in the counter mode. Due to its long output, one call (or two consecutive calls) would produce hundreds of addresses, thus minimizing the overhead. This approach does not give provable tradeoff bounds, but instead allows the analysis with the tradeoff algorithms suited for data-dependent addressing.

2.3.3 Implementing parallelism

As modern CPUs have several cores possibly available for hashing, it is tempting to use these cores to increase the bandwidth, the amount of filled memory, and the CPU load. The cores of the recent Intel CPU share the L3 cache and the entire memory, which both have large latencies (100 cycles and more). Therefore, the inter-processor communication should be minimal to avoid delays.

The simplest way to use p parallel cores is to compute and XOR p independent calls to H :

$$H'(P, S) = H(P, S, 0) \oplus H(P, S, 1) \oplus \dots \oplus H(P, S, p).$$

If a single call uses m memory units, then p calls use pm units. However, this method admits a trivial tradeoff: an adversary just makes p sequential calls to H using only m memory in total, which keeps the time-area product constant.

We suggest the following solution for p cores: the entire memory is split into p lanes of l equal slices each, which can be viewed as elements of a $(p \times l)$ -matrix $Q[i][j]$. Consider the class of schemes given by Equation (2.1). We modify it as follows:

- p invocations to H run in parallel on the first column $Q[*][0]$ of the memory matrix. Their indexing functions refer to their own slices only;
- For each column $j > 0$, l invocations to H continue to run in parallel, but the indexing functions now may refer not only to their own slice, but also to all jp slices of previous columns $Q[*][0], Q[*][1], \dots, Q[*][j-1]$.
- The last blocks produced in each slice of the last column are XORed.

This idea is easily implemented in software with p threads and l joining points. It is easy to see that the adversary can use less memory when computing the last column, for instance by computing the slices sequentially and storing only the slice which is currently computed. Then his time is multiplied by $(1 + \frac{p-1}{l})$, whereas the memory use is multiplied by $(1 - \frac{p-1}{pl})$, so the time-area product is modified as

$$AT_{new} = AT \left(1 - \frac{p-1}{pl}\right) \left(1 + \frac{p-1}{l}\right).$$

For $2 \leq p, l \leq 10$ this value is always between 1.05 and 3. We have selected $l = 4$ as this value gives low synchronisation overhead while imposing time-area penalties on the adversary who reduces the memory even by the factor $3/4$. We note that values $l = 8$ or $l = 16$ could be chosen.

If the compression function is collision-resistant, then one may easily prove that block collisions are highly unlikely. However, we employ a weaker compression function, for which the following holds:

$$G(X, Y) = F(X \oplus Y),$$

which is invariant to swap of inputs and is not collision-free. We take special care to ensure that the mode of operation does not allow such collisions by introducing additional rule:

- First block of a segment can not refer to the last block of any segment in the previous slice.

We prove that block collisions are unlikely under reasonable conditions on F in Section 2.4.4.

2.3.4 Compression function design

Overview

In contrast to attacks on regular hash functions, the adversary does not control inputs to the compression function G in our scheme. Intuitively, this should relax the cryptographic properties required from the compression function and allow for a faster primitive. To avoid being the bottleneck, the compression function ideally should be on par with the performance of `memcpy()` or similar function, which may run at 0.1 cycle per byte or even faster. This much faster than ordinary stream ciphers or hash functions, but we might not need strong properties of those primitives.

However, we first have to determine the optimal block size. When we request a block from a random location in the memory, we most likely get a cache miss. The first bytes would arrive at the CPU from RAM within at best 10 ns, which accounts for 30 cycles. In practice, the latency of a single load instruction may reach 100 cycles and more. However, this number can be amortized if we request a large block of sequentially stored bytes. When the first bytes are requested, the CPU stores the next ones in the L1 cache, automatically or using the `prefetch` instruction. The data from the L1 cache can be loaded as fast as 64 bytes per cycle on the Haswell architecture, though we did not manage to reach this speed in our application.

Therefore, the larger the block is, the higher the throughput is. We have made a series of experiments with a non-cryptographic compression function, which does little beyond simple XOR of its inputs, and achieved the performance of around 0.7 cycles per byte per core with block sizes of 1024 bits and larger.

Design criteria

Let us fix the block size t and figure out the design criteria. It appears that collision/preimage resistance and their weak variants are overkill as a design criteria for the compression function F . We recall, however, that the adversary is motivated by reducing the time-area product. Let us consider the following structure of the compression function $F(X, Y)$, where X and Y are input blocks:

- The input blocks of size t are divided into shorter subblocks of length t' (for instance, 128 bits) $X_0, X_1, X_2, \dots$ and $Y_0, Y_1, Y_2, \dots$
- The output block Z is computed subblockwise:

$$\begin{aligned} Z_0 &= G(X_0, Y_0); \\ Z_i &= G(X_i, Y_i, Z_{i-1}), \quad i > 0. \end{aligned}$$

This scheme resembles the duplex authenticated encryption mode, which is secure under certain assumptions on G . However, it is totally insecure against tradeoff adversaries, as shown below.

Attack on the iterative compression function. Suppose that an adversary computes $Z = F(X, Y)$ but Y is not stored. Suppose that Y is a tree function of stored elements of depth D . The adversary starts with computing Z_0 , which requires only Y_0 . In turn, $Y_0 = G(X'_0, Y'_0)$ for some X', Y' . Therefore, the adversary computes the tree of the same depth D , but with the function G instead of F . Z_1 is then a tree function of depth $D + 1$, Z_2 of depth $D + 2$, etc. In total, the recomputation takes $(D + s)L_G$ time, where s is the number of subblocks and L_G is the latency of G . This should be compared to the full-space implementation, which takes time sL_G . Therefore, if the memory is reduced by the factor q , then the time-area product is changed as

$$AT_{new} = \frac{D(q) + s}{sq} AT.$$

Therefore, if

$$D(q) \leq s(q - 1), \tag{2.2}$$

the adversary wins.

One may think of using the $Z_{m-1}[l - 1]$ as input to computing $Z_0[l]$. Clearly, this changes little in adversary's strategy, who could simply store all Z_{m-1} , which is feasible for large m . In concrete proposals, s can be 64, 128, 256 and even larger.

We conclude that F with an iterative structure is insecure. We note that this attack applies also to other PHC candidates with iterative compression function. Table 2.1 and Equation 2.2 suggests that it allows to reduce the memory by the factor of 12 or even higher while still reducing the area-time product.

Our approach We formulate the following design criteria:

- *The compression function must require about t bits of storage (excluding inputs) to compute any output bit.*
- *Each output byte of F must be a nonlinear function of all input bytes, so that the function has differential probability below certain level, for example $\frac{1}{4}$.*

These criteria ensure that the attacker is unable to compute an output bit using only a few input bits or a few stored bits. Moreover, the output bits should not be (almost) linear functions of input bits, as otherwise the function tree would collapse.

We have not found any generic design strategy for such large-block compression functions. It is difficult to maintain diffusion on large memory blocks due to the lack of CPU instructions that interleave many registers at once. A naive approach would be to apply a linear transformation with certain branch number. However, even if we operate on 16-byte registers, a 1024-byte block would consist of 64 elements. A 64×64 -matrix would require 32 XORs per register to implement, which gives a penalty about 2 cycles per byte.

Instead, we propose to build the compression function on the top of a transformation P that already mixes several registers. We apply P in parallel (having a P-box), then shuffle the output registers and apply it again. If P handles p registers, then the compression function may transform a block of p^2 registers with 2 rounds of P-boxes. We do not have to manually shuffle the data, we just change the

inputs to P-boxes. As an example, an implementation of the Blake2b [5] permutation processes 8 128-bit registers, so with 2 rounds of Blake2b we can design a compression function that mixes the 8192-bit block. We stress that this approach is not possible with dedicated AES instructions. Even though they are very fast, they apply only to the 128-bit block, and we still have to diffuse its content across other blocks.

2.3.5 User-controlled parameters

We have made a number of design choices, which we consider optimal for a wide range of applications. Some parameters can be altered, some should be kept as is. We give a user full control over:

- Amount M of memory filled by algorithm. This value, evidently, depends on the application and the environment. There is no "insecure" value for this parameter, though clearly the more memory the better.
- Number T of passes over the memory. The running time depends linearly on this parameter. We expect that the user chooses this number according to the time constraints on the application. Again, there is no "insecure value" for T .
- Degree d of parallelism. This number determines the number of threads used by an optimized implementation of Argon2. We expect that the user is restricted by a number of CPU cores (or half-cores) that can be devoted to the hash function, and chooses d accordingly (double the number of cores).
- Length of password/message, salt/nonce, and tag (except for some low, insecure values for salt and tag lengths).

We allow to choose another compression function G , hash function H , block size b , and number of slices l . However, we do not provide this flexibility in a reference implementation as we guess that the vast majority of the users would prefer as few parameters as possible.

2.4 Security analysis

2.4.1 Security of single-pass schemes

We consider the following types of indexing functions:

- Independent of the password and salt, but possibly dependent on other public parameters (*data-independent*). Thus the addresses can be calculated by the adversaries. Therefore, if the dedicated hardware can handle parallel memory access, the adversary can prefetch the data from the memory. Moreover, if she implements a time-space tradeoff, then the missing blocks can be also precomputed without losing time. In order to maximize the computational penalties, the designers proposed various formulas for indexing functions [10, 15], but several of them were found weak and admitting easy tradeoffs.
- Dependent on the password (*data-dependent*). A frequent choice is to use some bits of the previous block: $\phi_2(j) = g(B[j-1])$. This choice prevents the adversary from prefetching and precomputing missing data. The adversary figures out what he has to recompute only at the time the element is needed. If an element is recomputed as a tree of F calls of average depth D , then the total processing time is multiplied by D . However, this method is vulnerable to side-channel attacks, as timing information may help to filter out password guesses at the early stage.
- Hybrid schemes, where the first phase uses a data-independent addressing, and next phases use a data-dependent addressing. The side-channel attacks become harder to mount, since an adversary still has to run the first phase to filter out passwords. However, this first phase is itself vulnerable to time-space tradeoffs, as mentioned above.

In the case of *data-dependent* schemes, the adversary can reduce the time-area product if the time penalty due to the recomputation is smaller than the memory reduction factor. The time penalty is determined by the depth D of the recomputation tree, so the adversary wins as long as

$$D(q) \leq q.$$

In contrast, the cracking cost for *data-independent* schemes, expressed as the time-area product, is easy to reduce thanks to tradeoffs. The total area decreases until the area needed to host multiple cores for recomputation matches the memory area, whereas the total time remains stable until the total bandwidth required by the parallelized recomputations exceeds the architecture capabilities.

Let us elaborate on the first condition. When we follow some tradeoff strategy and reduce the memory by the factor of q , the total number of calls to G increases by the factor $C(q)$. Suppose that the logic for G takes A_{core} of area (measured, say, in mm^2), and the memory amount that we consider (say, 1 GB), takes A_{memory} of area. The adversary reduces the total area as long as:

$$C(q)A_{core} + A_{memory}/q \leq A_{memory}.$$

The maximum bandwidth Bw_{max} is a hypothetical upper bound on the memory bandwidth on the adversary's architecture. Suppose that for each call to G an adversary has to load $R(q)$ blocks from the memory on average, where q is the memory reduction factor. Therefore, the adversary can keep the execution time the same as long as

$$R(q)Bw \leq Bw_{max},$$

where Bw is the bandwidth achieved by a full-space implementation.

Lemma 1.

$$R(q) = C(q).$$

This lemma is proved in Section 2.7.2.

2.4.2 Ranking tradeoff attack

To figure out the costs of the ASIC-equipped adversary, we first need to calculate the time-space tradeoffs for our class of hashing schemes. To the best of our knowledge, the first generic tradeoffs attacks were reported in [7], and they apply to both data-dependent and data-independent schemes. The idea of the ranking method [7] is as follows. When we generate a memory block $X[l]$, we make a decision, to store it or not. If we do not store it, we calculate the access complexity of this block — the number of calls to F needed to compute the block, which is based on the access complexity of $X[l-1]$ and $X[\phi(l)]$. The detailed strategy is as follows:

1. Select an integer q (for the sake of simplicity let q divide T).
2. Store $X[kq]$ for all k ;
3. Store all r_i and all access complexities;
4. Store the T/q highest access complexities. If $X[i]$ refers to a vertex from this top, we store $X[i]$.

The memory reduction is a probabilistic function of q . We reimplemented the algorithm and applied to the scheme 2.1 where the addresses are randomly generated. We obtained the results in Table 2.1. Each recomputation is a tree of certain depth, also given in the table.

We conclude that for data-dependent one-pass schemes the adversary is always able to reduce the memory by the factor of 4 and still keep the time-area product the same.

Memory fraction ($1/q$)	$\frac{1}{2}$	$\frac{1}{3}$	$\frac{1}{4}$	$\frac{1}{5}$	$\frac{1}{6}$	$\frac{1}{7}$	$\frac{1}{8}$	$\frac{1}{9}$	$\frac{1}{10}$	$\frac{1}{11}$
Computation&Read penalty $C(q)$	1.71	2.95	6.3	16.6	55	206	877	4423	$2^{14.2}$	$2^{16.5}$
Depth penalty ($D(q)$)	1.7	2.5	3.8	5.7	8.2	11.5	15.7	20.8	26.6	32.8

Table 2.1: Time and computation penalties for the ranking tradeoff attack for random addresses.

2.4.3 Multi-pass schemes

If the defender has more time than needed to fill the available memory, then he can run several passes on the memory. Also some designers decided to process memory several times to get better time-space tradeoffs. Let us figure out how the adversary's costs are affected in this case.

Suppose we make K passes with T iterations each following the scheme (2.1), so that after the first pass any address in the memory may be used. Then this is equivalent to running a single pass with KT iterations such that $\phi(j) \geq j - T$. The time-space tradeoff would be the same as in a single pass with T iterations and additional condition

$$\phi(j) \geq j - \frac{T}{K}.$$

We have applied the ranking algorithm (Section 2.4.2) and obtained the results in Tables 2.2, 2.3. We conclude that for the data-dependent schemes using several passes does increase the time-area product for the adversary who uses tradeoffs. Indeed, suppose we run a scheme with memory A with one pass for time T , or on $A/2$ with 2 passes. If the adversary reduces the memory to $A/6$ GB (i.e. by the factor of 6) for the first case, the time grows by the factor of 8.2, so that the time-area product is $1.35AT$. However, if in the second setting the memory is reduced to $A/6$ GB (i.e. by the factor of 3), the time grows by the factor of 14.3, so that the time-area product is $2.2AT$. For other reduction factors the ratio between the two products remains around 2.

Nevertheless, we do not immediately argue for the prevalence of multi-pass schemes, since it can be possible that new tradeoff algorithms change their relative strength.

Memory fraction ($1/q$)	$\frac{1}{2}$	$\frac{1}{3}$	$\frac{1}{4}$	$\frac{1}{5}$	$\frac{1}{6}$
1 pass	1.7	3	6.3	16.6	55
2 passes	15	410	19300	2^{20}	2^{25}
3 passes	3423	2^{22}	2^{32}		

Table 2.2: Computation/read penalties for the ranking tradeoff attack.

Memory fraction ($1/q$)	$\frac{1}{2}$	$\frac{1}{3}$	$\frac{1}{4}$	$\frac{1}{5}$	$\frac{1}{6}$
1 pass	1.7	2.5	3.8	5.7	8.2
2 passes	5.7	14.3	28.8	49	75
3 passes	20.7	56	103	—	—

Table 2.3: Depth penalties for the ranking tradeoff attack.

2.4.4 Security of Argon2 to generic attacks

Now we consider preimage and collision resistance of both versions of **Argon2**. Variable-length inputs are prepended with their lengths, which shall ensure the absence of equal input strings. Inputs are processed by a cryptographic hash function, so no collisions should occur at this stage.

Internal collision resistance. The compression function G is not claimed to be collision resistant, so it may happen that distinct inputs produce identical outputs. Recall that G works as follows:

$$G(X, Y) = P(Z) \oplus (Z), \quad Z = X \oplus Y.$$

where P is a permutation based on the 2-round Blake2b permutation. Let us prove that all Z are different under certain assumptions.

Theorem 1. *Let Π be Argon2d or Argon2i with d lanes, s slices, and t passes over memory. Assume that*

- $P(Z) \oplus Z$ is collision-resistant, i.e. it is hard to find a, b such that $P(a) \oplus a = P(b) \oplus b$.

- $P(Z) \oplus Z$ is 4-generalized-birthday-resistant, i.e. it is hard to find distinct a, b, c, d such that $P(a) \oplus P(b) \oplus P(c) \oplus P(d) = a \oplus b \oplus c \oplus d$.

Then all the blocks $B[i]$ generated in those t passes are different.

Proof. By specification, the value of Z is different for the first two blocks of each segment in the first slice in the first pass. Consider the other blocks.

Let us enumerate the blocks according to the moment they are computed. Within a slice, where segments can be computed in parallel, we enumerate lane 0 fully first, then lane 1, etc.. Slices are then computed and enumerated sequentially. Suppose the proposition is wrong, and let $(B[a], B[b])$ be a block collision such that $x < y$ and y is the smallest among all such collisions. As $F(Z) \oplus Z$ is collision resistant, the collision occurs in Z , i.e.

$$Z_x = Z_y.$$

Let r_x, r_y be reference block indices for $B[x]$ and $B[y]$, respectively, and let p_x, p_y be previous block indices for $B[x], B[y]$. Then we get

$$B[r_x] \oplus B[p_x] = B[r_y] \oplus B[p_y].$$

As we assume 4-generalized-birthday-resistance, some arguments are equal. Consider three cases:

- $r_x = p_x$. This is forbidden by the rule 3 in Section 2.1.3.
- $r_x = r_y$. We get $B[p_x] = B[p_y]$. As $p_x, p_y < y$, and y is the smallest yielding such a collision, we get $p_x = p_y$. However, by construction $p_x \neq p_y$ for $x \neq y$.
- $r_x = p_y$. Then we get $B[r_y] = B[p_x]$. As $r_y < y$ and $p_x < x < y$, we obtain $r_y = p_x$. Since $p_y = r_x < x < y$, we get that x and y are in the same , we have two options:
 - p_y is the last block of a segment. Then y is the first block of a segment in the next slice. Since r_x is the last block of a segment, and $x < y$, x must be in the same slice as y , and x can not be the first block in a segment by the rule 4 in Section 2.1.3. Therefore, $r_y = p_x = x - 1$. However, this is impossible, as r_y can not belong to the same slice as y .
 - p_y is not the last block of a segment. Then $r_x = p_y = y - 1$, which implies that $r_x \geq x$. The latter is forbidden.

Thus we get a contradiction in all cases. This ends the proof. $\square$

The compression function G is not claimed to be collision resistant nor preimage-resistant. However, as the attacker has no control over its input, the collisions are highly unlikely. We only take care that the starting blocks are not identical by producing the first two blocks with a counter and forbidding to reference from the memory the last block as (pseudo)random.

Argon2d does not overwrite the memory, hence it is vulnerable to garbage-collector attacks and similar ones, and is not recommended to use in the setting where these threats are possible. **Argon2i** with 3 passes overwrites the memory twice, thus thwarting the memory-leak attacks. Even if the entire working memory of **Argon2i** is leaked after the hash is computed, the adversary would have to compute two passes over the memory to try the password.

2.4.5 Security of Argon2 to tradeoff attacks

Time and computational penalties for 1-pass **Argon2d** are given in Table 2.1. It suggests that the adversary can reduce memory by the factor of 4 while keeping the time-area product the same.

Argon2i is more vulnerable to tradeoff attacks due to its data-independent addressing scheme. We apply the ranking algorithm to 3-pass **Argon2i** to calculate time and computational penalties. Table 2.2 demonstrates that the memory reduction by the factor of 3 already gives the computational penalty of around 2^{14} . The 2^{14} Blake2b cores would take more area than 1 GB of RAM (Section 2.3.1), thus prohibiting the adversary to further reduce the time-area product. We conclude that the time-area product cost for **Argon2d** can be reduced by 3 at best.

Processor	Threads	Argon2d (1 pass)		Argon2i (3 passes)	
		Cycles/Byte	Bandwidth (GB/s)	Cycles/Byte	Bandwidth (GB/s)
Core i7-4600U	1	1.7	2.5	4.8	2.6
Core i7-4500U	1	1.1	3.3	3.4	3.2
Core i7-4600U	2	1	4.2	2.8	4.5
Core i7-4500U	2	0.7	5.1	2.1	5.1
Core i7-4600U	4	0.9	4.9	2.3	5.5
Core i7-4500U	4	0.6	6	1.6	6.8
Core i7-4600U	8	0.8	5	2.3	5.4
Core i7-4500U	8	0.6	6	1.6	6.8

Table 2.4: Speed and memory bandwidth of Argon2(d/i). Core i7-4500U — Intel Haswell 1.8 GHz, 4 cores; Core i7-4600U — Intel Haswell 2.1 GHz, 4 cores

2.5 Performance

2.5.1 x86 architecture

To optimize the data load and store from/to memory, the memory that will be processed has to be aligned on 16-byte boundary when loaded/stored into/from 128-bit registers and on 32-byte boundary when loaded/stored into/from 256-bit registers. If the memory is not aligned on the specified boundaries, then each memory operation may take one extra CPU cycle, which may cause consistent penalties for many memory accesses.

The results presented are obtained using the `gcc 4.8.2` compiler with the following options: `-m64 -mavx -std=c++11 -pthread -O3`. The cycle count value was measured using the `__rdtscp` Intel intrinsic C function which inlines the `RDTSCP` assembly instruction that returns the 64-bit Time Stamp Counter (TSC) value. The instruction waits for previous instruction to finish and then is executed, but meanwhile the next instructions may begin before the value is read [14]. Although this shortcoming, we used this method because it is the most reliable handy method to measure the execution time and also it is widely used in other cryptographic operations benchmarking.

2.6 Applications

Argon2d is optimized for settings where the adversary does not get regular access to system memory or CPU, i.e. he can not run side-channel attacks based on the timing information, nor he can recover the password much faster using garbage collection [9]. These settings are more typical for backend servers and cryptocurrency minings. For practice we suggest the following settings:

- Cryptocurrency mining, that takes 0.1 seconds on a 2 GHz CPU using 1 core — Argon2d with 2 lanes and 250 MB of RAM;
- Backend server authentication, that takes 0.5 seconds on a 2 GHz CPU using 4 cores — Argon2d with 8 lanes and 4 GB of RAM.

Argon2i is optimized for more dangerous settings, where the adversary possibly can access the same machine, use its CPU or mount cold-boot attacks. We use three passes to get rid entirely of the password in the memory. We suggest the following settings:

- Key derivation for hard-drive encryption, that takes 3 seconds on a 2 GHz CPU using 2 cores — Argon2i with 4 lanes and 6 GB of RAM;
- Frontend server authentication, that takes 0.5 seconds on a 2 GHz CPU using 2 cores — Argon2i with 4 lanes and 1 GB of RAM.

2.7 Other details

2.7.1 Permutation $\mathcal{P}$

Permutation $\mathcal{P}$ is based on the round function of Blake2b and works as follows. Its 8 16-byte inputs $S_0, S_1, \dots, S_7$ are viewed as a 4×4 -matrix of 64-bit words, where $S_i = (v_{2i+1} || v_{2i})$:

$$\begin{pmatrix} v_0 & v_1 & v_2 & v_3 \\ v_4 & v_5 & v_6 & v_7 \\ v_8 & v_9 & v_{10} & v_{11} \\ v_{12} & v_{13} & v_{14} & v_{15} \end{pmatrix}$$

Then we do

$$\begin{matrix} G(v_0, v_4, v_8, v_{12}) & G(v_1, v_5, v_9, v_{13}) & G(v_2, v_6, v_{10}, v_{14}) & G(v_3, v_7, v_{11}, v_{15}) \\ G(v_0, v_5, v_{10}, v_{15}) & G(v_1, v_6, v_{11}, v_{12}) & G(v_2, v_7, v_8, v_{13}) & G(v_3, v_4, v_9, v_{14}), \end{matrix}$$

where G applies to (a, b, c, d) as follows:

$$\begin{aligned} a &\leftarrow a + b; \\ d &\leftarrow (d \oplus a) \ggg 32; \\ c &\leftarrow c + d; \\ b &\leftarrow (b \oplus c) \ggg 24; \\ a &\leftarrow a + b; \\ d &\leftarrow (d \oplus a) \ggg 16; \\ c &\leftarrow c + d; \\ b &\leftarrow (b \oplus c) \ggg 63; \end{aligned} \tag{2.3}$$

Here $+$ are additions modulo 2^{64} and $\ggg$ are 64-bit rotations to the right.

2.7.2 Proof of Lemma 1

Proof. Let A_j be the computational complexity of recomputing $M[j]$. If $M[j]$ is stored, then $A_j = 0$. When we have to compute a new block $M[i]$, then the computational complexity C_i of computing $M[i]$ (measured in calls to F) is calculated as

$$C_i = A_{\phi_2(i)} + 1.$$

and the total computational penalty is calculated as

$$C(q) = \frac{\sum_{i < T} (A_{\phi_2(i)} + 1)}{T}.$$

Let R_j be the total number of blocks to be read from the memory in order to recompute $M[j]$. The total bandwidth penalty is calculated as

$$R(q) = \frac{\sum_{i < T} R_{\phi_2(i)}}{T}.$$

Let us prove that

$$R_j = A_j + 1. \tag{2.4}$$

by induction.

- We store $M[0]$, so for $j = 0$ we have $R_0 = 1$ and $A_0 = 0$.
- If $M[j]$ is stored, then we read it and make no call to F , i.e.

$$A_j = 0; \quad R_j = 1.$$

- If $M[j]$ is not stored, we have to recompute $M[j-1]$ and $M[\phi_2(j)]$:

$$A_j = A_{j-1} + A_{\phi_2(j)} + 1 = R_{j-1} - 1 + R_{\phi_2(j)} - 1 + 1 = (R_{j-1} + R_{\phi_2(j)}) - 1 = R_j - 1.$$

The last equation follows from the fact that the total amount of reads for computing $M[j]$ is the sum of necessary reads for $M[j-1]$ and $M[\phi_2(j)]$.

Therefore, we get

$$C(q) = \frac{\sum_{i < T} (A_{\phi_2(i)} + 1)}{T} = \frac{\sum_{i < T} R_{\phi_2(i)}}{T} = R(q).$$

□

Chapter 3

Update

Here we summarize an optional update to Argon2. Following discussions in PHC forum we decided to further improve circuit depth and tradeoff resistance of Argon. We appreciate feedback from the PHC community on whether to make them default.

3.1 Indexing

3.1.1 Description

Originally Argon2 selected the reference block pseudo-randomly among the previously generated blocks (except for segments currently being produced in the other lanes). We implement smarter non-linear indexing similar to that used in TwoCats [8] and yescrypt [18] in order to flatten the memory usage over-time.

Now the 32-bit value L used to refer to the block is treated as follows:

- The most significant 8 bits L_0 determine the lane r from which the block is taken:

$$r \leftarrow L_0 \bmod p,$$

where p is the number of lanes.

- The least significant 24 bits L_1 determine the block index in lane r . The available blocks are enumerated according to their creation date from 0 to $R - 1$. Then the block number ϕ is defined as

$$\phi \leftarrow \lceil ((L_1 \bmod R)/R)^\alpha R \rceil$$

Therefore, the blocks are referenced with different probability. For $\alpha > 1$ the probability grows as a polynomial of degree $(\alpha - 1)$ of the relative position of the block. We selected $\alpha = 2$ as a default option, and allow $\alpha = 1, 3, 4, 5$ as other possible settings (in TwoCats $\alpha = 3$).

3.1.2 Rationale

We tried to choose the α which would maximize the adversary's costs if he applies the tradeoff based on the ranking method. We also attempted to make the reference block distribution close to uniform, so that each memory block is referenced similar number of times.

For each $1 \leq \alpha \leq 5$ with step 0.1 we applied the ranking method with sliding window and selected the best available tradeoffs. We obtained a set of latency penalties $\{LP_\alpha(q)\}$ and computational penalties $\{CP_\alpha(q)\}$ for $1 < q < 30$. We also calculated the reference block distribution for all possible α . We considered two possible metrics:

1. Minimum time-area product

$$AT_\alpha = \min_q \{q \cdot LP_\alpha(q)\}.$$

2. Maximum memory reduction which reduces the time-area product compared to the original:

$$q_\alpha = \max_q \{q \mid LP_\alpha(q) < q\}.$$

3. The goodness-of-fit value of the reference block distribution w.r.t. the uniform distribution with n bins:

$$\chi^2 = \sum_i \frac{(p_i - \frac{1}{n})^2}{\frac{1}{n}},$$

where p_i is the average probability of the block from i -th bin to be referenced. For example, if $p_3 = 0.2$, $n = 10$ and there are 1000 blocks, then blocks from 201 to 300 are referenced $1000 \cdot 0.2 = 200$ times throughout the computation.

We got the following results for $n = 10$:

α	AT_α	q_α	χ^2
1	0.78	3.95	0.89
1.2	0.77	3.65	0.71
1.4	0.76	3.42	0.57
1.6	0.74	3.33	0.5
1.8	0.73	3.25	0.4
2	0.72	3.2	0.35
2.2	0.71	3.3	0.31
2.4	0.7	3.35	0.28
2.6	0.69	3.31	0.24
2.8	0.68	3.41	0.21
3	0.67	3.48	0.2
3.2	0.66	3.52	0.18
3.4	0.65	3.58	0.15
3.6	0.64	3.77	0.14
3.8	0.64	3.76	0.13
4	0.63	3.9	0.13
5	0.59	4.38	0.09

We conclude that the time-area product achievable by the attacker slowly decreases as α grows. However, the difference between $\alpha = 1$ and $\alpha = 5$ is only the factor of 1.3. We also see that the time-area product can be kept below the original up to $q = 3.2$ for $\alpha = 2$, whereas for $\alpha = 4$ and $\alpha = 1$ such q is close to 4. To avoid floating-point computations, we restrict to integer α . Thus the optimal values are $\alpha = 2$ and $\alpha = 3$, where the former is slightly better in the first two metrics.

However, if we consider the reference block uniformity, the situation favors larger α considerably. We see that the χ^2 value is decreased by the factor of 2.5 when going from $\alpha = 1$ to $\alpha = 2$, and by the factor of 1.8 further to $\alpha = 3$. In concrete probabilities (see also Figure 3.1), the first 20% of blocks accumulate 40% of all reference hits for $\alpha = 2$ and 32% for $\alpha = 3$ (23.8% vs 19.3% hit for the first 10% of blocks).

To summarize, $\alpha = 2$ and $\alpha = 3$ both are better against one specific attacker and slightly worse against the other. We take $\alpha = 2$ as the value that minimizes the AT gain, as we consider this metric more important.

Memory fraction ($1/q$)	$\frac{1}{2}$	$\frac{1}{3}$	$\frac{1}{4}$	$\frac{1}{5}$	$\frac{1}{6}$
$\alpha = 1$	1.6	2.9	7.3	16.4	59
$\alpha = 2$	1.5	4	20.2	344	4700
$\alpha = 3$	1.4	4.3	28.1	1040	2^{17}

Table 3.1: Computational penalties for the ranking tradeoff attack with a sliding window, 1 pass.

3.2 New compression function

3.2.1 Design

We replace the Blake2b round function in permutation $\mathcal{P}$ with its modification BlaMka [16], where the modular additions in G are combined with 32-bit multiplications (that is the only difference):

Memory fraction ($1/q$)	$\frac{1}{2}$	$\frac{1}{3}$	$\frac{1}{4}$	$\frac{1}{5}$	$\frac{1}{6}$
$\alpha = 1$	1.6	2.5	4	5.8	8.7
$\alpha = 2$	1.5	2.6	5.4	10.7	17
$\alpha = 3$	1.3	2.5	5.3	10.1	18

Table 3.2: Depth penalties for the ranking tradeoff attack with a sliding window, 1 pass.

$$\begin{aligned}
a &\leftarrow a + b + \underbrace{2 * a_L * b_L}_{new}; \\
d &\leftarrow (d \oplus a) \ggg 32; \\
c &\leftarrow c + d + \underbrace{2 * c_L * d_L}_{new}; \\
b &\leftarrow (b \oplus c) \ggg 24; \\
a &\leftarrow a + b + \underbrace{2 * a_L * b_L}_{new}; \\
d &\leftarrow (d \oplus a) \ggg 16; \\
c &\leftarrow c + d + \underbrace{2 * c_L * d_L}_{new}; \\
b &\leftarrow (b \oplus c) \ggg 63;
\end{aligned} \tag{3.1}$$

Here x_L is the 64-bit integer x truncated to the 32 least significant bits.

3.2.2 Rationale

Our motivation was to increase the circuit depth (and thus the running time) of a potential ASIC implementation while having roughly the same running time on CPU thanks to parallelism and pipelining. Extra multiplications in the scheme serve well, as the best addition-based circuits for multiplication have latency about 4-5 times the addition latency for 32-bit multiplication (or roughly $\log_n$ for n -bit multiplication).

As a result, any output 64-bit word of $\mathcal{P}$ is implemented by a chain of additions, multiplications, XORs, and rotations. The shortest possible chain for the 1 KB-block (e.g, from v_0 to v_0) consists of 12 MULs, 12 XORs, and 12 rotations.

3.3 Performance

In Table 3.3 we present the benchmarks of the new version with $\alpha = 2$. The new indexing rule does not affect the performance much, though with high number of threads it reaches 0.55 cycles per byte. However, the multiplications in the compression function impose a penalty. Interestingly, the penalty is more noticeable for 1-thread version (1.6 vs 1.1 cpb, i.e. -50% of speed) and almost negligible for 4- and 8-thread versions (0.7 vs 0.6 cpb, i.e. about 13% decrease). This clearly states that the memory bus becomes the bottleneck for high levels of parallelism.

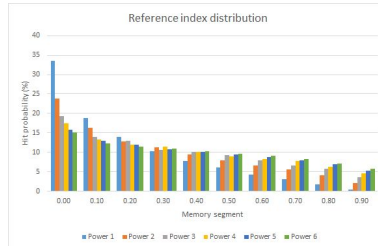


Figure 3.1: Access frequency for different memory segments (10%-buckets) and different exponents (from $\alpha = 1$ to $\alpha = 5$) in the indexing functions.

Processor	Threads	Argon2d (1 pass)		Argon2i (3 passes)	
		Cycles/Byte	Bandwidth (GB/s)	Cycles/Byte	Bandwidth (GB/s)
Core i7-4500U	1	1.6	2.2	4.7	2.6
Core i7-4500U	2	1.0	3.6	2.8	4.5
Core i7-4500U	4	0.7	5.1	2	5.4
Core i7-4500U	8	0.7	5.1	1.9	5.8

Table 3.3: Speed and memory bandwidth of Argon2(d/i) measured on 1 GB memory filled. Core i7-4500U
— Intel Haswell 1.8 GHz, 4 cores

Chapter 4

Change log

4.1 v1.2 of Argon2 – 21th June, 2015

New version (update): new indexing rule, new compression function.

4.2 v1.1 of Argon2 – 4th February, 2015

New indexing rule added to avoid collision with a proof.

Bibliography

- [1] Micron power calculator. <http://www.micron.com/products/support/power-calc>.
- [2] *NIST: AES competition*, 1998. <http://csrc.nist.gov/archive/aes/index.html>.
- [3] *NIST: SHA-3 competition*, 2007. <http://csrc.nist.gov/groups/ST/hash/sha-3/index.html>.
- [4] *Litecoin - Open source P2P digital currency*, 2011. <https://litecoin.org/>.
- [5] Jean-Philippe Aumasson, Samuel Neves, Zooko Wilcox-O’Hearn, and Christian Winnerlein. BLAKE2: simpler, smaller, fast as MD5. In *ACNS’13*, volume 7954 of *Lecture Notes in Computer Science*, pages 119–135. Springer, 2013.
- [6] Daniel J. Bernstein and Tanja Lange. Non-uniform cracks in the concrete: The power of free precomputation. In *ASIACRYPT’13*, volume 8270 of *Lecture Notes in Computer Science*, pages 321–340. Springer, 2013.
- [7] Alex Biryukov, Dmitry Khovratovich, and Johann Grozschädl. Tradeoff cryptanalysis of password hashing schemes. Technical report, 2014. available at <https://www.cryptolux.org/images/5/57/Tradeoffs.pdf>.
- [8] Bill Cox. Twocats (and skinnycat): A compute time and sequential memory hard password hashing scheme. Technical report, 2014. available at <https://password-hashing.net/submissions/specs/TwoCats-v0.pdf>.
- [9] Christian Forler, Eik List, Stefan Lucks, and Jakob Wenzel. Overview of the candidates for the password hashing competition – and their resistance against garbage-collector attacks. *Cryptology ePrint Archive, Report 2014/881*, 2014. <http://eprint.iacr.org/>.
- [10] Christian Forler, Stefan Lucks, and Jakob Wenzel. Catena: A memory-consuming password scrambler. *IACR Cryptology ePrint Archive, Report 2013/525*. to appear in *Asiacrypt’14*.
- [11] Bharan Giridhar, Michael Cieslak, Deepankar Duggal, Ronald G. Dreslinski, Hsing Min Chen, Robert Patti, Betina Hold, Chaitali Chakrabarti, Trevor N. Mudge, and David Blaauw. Exploring DRAM organizations for energy-efficient and resilient exascale memories. In *International Conference for High Performance Computing, Networking, Storage and Analysis (SC 2013)*, pages 23–35. ACM, 2013.
- [12] Frank Gürkaynak, Kris Gaj, Beat Muheim, Ekawat Homsirikamol, Christoph Keller, Marcin Rogawski, Hubert Kaeslin, and Jens-Peter Kaps. Lessons learned from designing a 65nm ASIC for evaluating third round SHA-3 candidates. In *Third SHA-3 Candidate Conference*, March 2012.
- [13] Martin E Hellman. A cryptanalytic time-memory trade-off. *Information Theory, IEEE Transactions on*, 26(4):401–406, 1980.
- [14] Intel Corporation. *Intel® 64 and IA-32 Architectures Software Developer’s Manual*. Number 325462-052US. September 2014.
- [15] Marcos A. Simplicio Jr, Leonardo C. Almeida, Ewerton R. Andrade, Paulo C. F. dos Santos, and Paulo S. L. M. Barreto. The Lyra2 reference guide, version 2.3.2 (april 4th, 2013). Technical report, 2014. available at <https://password-hashing.net/submissions/specs/Lyra2-v1.pdf>.
- [16] Marcos A. Simplicio Jr., Leonardo C. Almeida, Ewerton R. Andrade, Paulo C. F. dos Santos, and Paulo S. L. M. Barreto. Lyra2: Password hashing scheme with improved security against time-memory trade-offs. *Cryptology ePrint Archive, Report 2015/136*, 2015. <http://eprint.iacr.org/>.

- [17] Colin Percival. Stronger key derivation via sequential memory-hard functions. 2009. <http://www.tarsnap.com/scrypt/scrypt.pdf>.
- [18] Alexander Peslyak. yescrypt - a password hashing competition submission. Technical report, 2014. available at <https://password-hashing.net/submissions/specs/yescrypt-v0.pdf>.
- [19] Thomas Ristenpart, Eran Tromer, Hovav Shacham, and Stefan Savage. Hey, you, get off of my cloud: exploring information leakage in third-party compute clouds. In *Proceedings of the 2009 ACM Conference on Computer and Communications Security, CCS 2009, Chicago, Illinois, USA, November 9-13, 2009*, pages 199–212, 2009.
- [20] Matthew Robshaw and Olivier Billet. *New stream cipher designs: the eSTREAM finalists*, volume 4986. Springer, 2008.
- [21] Bram Rooseleer, Stefan Cosemans, and Wim Dehaene. A 65 nm, 850 mhz, 256 kbit, 4.3 pj/access, ultra low leakage power memory using dynamic cell stability and a dual swing data link. *J. Solid-State Circuits*, 47(7):1784–1796, 2012.
- [22] Clark D. Thompson. Area-time complexity for VLSI. In *STOC’79*, pages 81–88. ACM, 1979.